

具身智能大模型

面试题库与精讲

Embodied AI & VLA Interview Handbook

12 大模块 • 78 道高频题 • 数理推导 + PyTorch 代码
从基础概念到 SOTA (RT / OpenVLA / π_0 / RDT)

整理: _____

版本 v1.1

目 录

starttoctoc

1 具身智能大模型 · 面试题库总览

方向: *Embodied AI* + 大模型 (VLA / 多模态 / 机器人基础模型) 用法: 先通览本题库, 标记薄弱模块; 再进入「逐题精讲」逐个攻克。标记: ★ 高频必考 | ☆ 进阶/加分 | ◆ 开放/研究思维题

模块地图

编号	模块	核心考点	精讲文件
M1	具身智能基础概念	定义、范式、与传统机器人/纯 LLM 的区别	逐 题 精 讲/01-基 础 概 念.md
M2	多模态大模型基础 (VLM)	视觉编码、对齐、注意力、tokenizer	逐 题 精 讲/02-多 模 态 基 础.md
M3	VLA 模型	架构、action head、动作离散化	逐 题 精 讲/03-VLA 模 型.md
M4	模仿学习与策略	BC、ACT、Diffusion Policy	逐题精讲/04-模仿学习与策略.md
M5	强化学习在具身	RLHF/在线 RL、sim 中的 RL、奖励设计	逐 题 精 讲/05-强 化 学 习.md
M6	世界模型与预测	world model、video prediction、规划	逐 题 精 讲/06-世 界 模 型.md
M7	数据、仿真与 Sim2Real	遥操作、数据飞轮、域随机化	逐 题 精 讲/07-数 据 与 仿 真.md
M8	感知与 3D 表示	点云、NeRF/3DGS、空间表征	逐 题 精 讲/08-感 知 与 3D.md
M9	规划、控制与系统	分层架构、实时性、安全	逐题精讲/09-规划控制系统.md
M10	经典论文与 SOTA	RT 系列、OpenVLA、 π^0 、RDT、Octo	逐 题 精 讲/10-经 典 论 文 SOTA.md
M11	工程与部署	推理加速、边缘部署、频率	逐 题 精 讲/11-工 程 部 署.md
M12	开放题与项目面	研究品味、项目深挖、反问	逐题精讲/12-开放题与项目面.md
附 A	数学基础速查	attention/对 比/扩 散/flow/VAE/RL/旋 转	深度补充/A-数学基础速查.md
附 B	核心算法代码	PyTorch 手写各核心 loss 与策略	深度补充/B-核心算法代码.md

深度补充是为了应对”推导一下””手写个 $loss$ ”类深挖，建议在过完对应模块后配套学。例：学完 $M4(Diffusion Policy)$ → 看附 A3 扩散数学 + 附 B3 代码。

M1 · 具身智能基础概念

1. ★ 什么是具身智能 (Embodied AI)? 它和传统机器人控制、和纯语言大模型相比，本质区别是什么?
2. ★ ”具身”为什么重要? 请从感知-行动闭环 (perception-action loop) 角度解释。
3. 摩拉维克悖论 (Moravec's Paradox) 是什么? 它对具身智能意味着什么?
4. 符号主义、联结主义、具身认知三条路线的区别? 具身智能属于哪一脉?
5. ★ 大模型时代，具身智能相比”互联网 AI (Internet AI)”面临的独特挑战有哪些? (数据、实时、安全、长尾)
6. 一个完整的具身智能体包含哪些模块? 请画出 sense→think→act 的 pipeline。
7. ◆ 你认为”通用机器人脑”目前最大的瓶颈是什么? 为什么?

M2 · 多模态大模型基础 (VLM)

1. ★ 视觉信息如何输入到大模型? 对比 CLIP、ViT-patch、Q-Former、cross-attention 几种方式。
2. ★ CLIP 的训练目标是什么? 对比学习 (contrastive) 为什么对具身有用?
3. 主流 VLM (LLaVA / BLIP-2 / Flamingo / Qwen-VL) 的视觉-语言对齐方式分别是怎样的?
4. 什么是 visual token? 图像 token 数量过多会带来什么问题，如何压缩?
5. ★ 为什么很多 VLA 直接复用预训练 VLM 作为 backbone? 带来什么先验?
6. 多模态对齐中的”模态鸿沟 (modality gap)”指什么?
7. ◆ 视频理解相比图像理解，对具身有什么额外价值和难点?

M3 · VLA 模型 (Vision-Language-Action)

1. ★ 什么是 VLA 模型? 输入输出分别是什么? 请描述其典型架构。
2. ★ 机器人动作 (连续、高维、高频) 如何被语言模型表示? 对比”动作离散化为 token” vs ”回归连续值” vs ”diffusion/flow head”。
3. ★ RT-2 的核心思想是什么? 它如何把动作变成 token 接到 VLM 上?
4. OpenVLA 相比 RT-2 做了哪些改进? 为什么能开源复现?
5. ★ action chunking 是什么? 为什么能缓解误差累积和抖动?
6. VLA 的动作输出频率通常受什么限制? 大模型推理慢如何与高频控制 (如 1kHz) 协调?
7. ★ $\pi 0$ (Pi-Zero) 用 flow matching 输出动作，相比离散 token 有什么优势?
8. ◆ 你会如何设计一个 VLA，使它能 zero-shot 泛化到没见过的物体/指令?

M4 · 模仿学习与策略学习

1. ★ 行为克隆 (BC) 的基本原理? 它最大的问题是什么 (distribution shift / compounding error)?
2. ★ DAgger 如何缓解 BC 的分布偏移? 代价是什么?
3. ★ Diffusion Policy 的核心思想? 为什么用扩散模型建模动作比直接回归好? (多模态分布)
4. ★ ACT (Action Chunking with Transformers) 的关键设计? CVAE 在其中起什么作用?
5. 模仿学习中”多模态动作分布”指什么? 为什么 MSE 回归会导致”取平均”失败?
6. ★ 如何评估一个模仿学习策略的好坏? 仿真成功率和真机成功率为何常常不一致?
7. ◆ 只有少量人类示范数据, 你如何提升策略的鲁棒性?

M5 · 强化学习在具身

1. ★ 强化学习和模仿学习在具身任务中各自的优劣? 什么时候该用哪个?
2. on-policy (PPO) vs off-policy (SAC) 在机器人中的取舍?
3. ★ 稀疏奖励问题如何解决? (reward shaping、HER、curriculum、demonstration)
4. ★ 为什么真机上做 RL 很难? sim 中训练再迁移的主要障碍是什么?
5. RLHF 的思想能否迁移到具身? 如何用偏好/人类反馈优化机器人策略?
6. ◆ 给定一个”把杯子放到架子上”的任务, 你如何设计奖励函数? 会踩哪些坑?

M6 · 世界模型与预测

1. ★ 什么是世界模型 (World Model)? 它在具身智能中扮演什么角色?
2. 基于模型的 RL (MBRL) 相比 model-free 的优势与风险?
3. ★ 用视频生成模型 (如 Sora 类) 做世界模型/策略, 思路是什么? 有什么局限?
4. Dreamer 系列的核心机制 (latent imagination) 是什么?
5. ◆ ”预测下一帧”和”预测下一个动作”哪个对机器人更本质? 谈谈你的看法。

M7 · 数据、仿真与 Sim2Real

1. ★ 具身智能的数据从哪来? 对比遥操作、人类视频、仿真、自主采集的优缺点。
2. ★ 什么是 Sim2Real gap? 主要来源有哪些 (动力学、渲染、传感器噪声)?
3. ★ 域随机化 (Domain Randomization) 的原理? 如何选择随机化的维度?
4. Open X-Embodiment 数据集的意义是什么? 跨本体 (cross-embodiment) 训练的难点?
5. ★ 数据飞轮 (data flywheel) 在机器人公司里怎么转起来?
6. 遥操作数据的常见质量问题有哪些? 如何清洗?
7. ◆ 给你 100 台机器人和有限预算, 你如何设计数据采集策略最大化策略性能?

M8 · 感知与 3D 表示

1. ★ 机器人为什么常需要 3D 表示而非纯 2D? 点云、深度图、voxel、3DGS 各自适用场景?
2. 6-DoF 抓取位姿估计的常见做法?
3. ★ NeRF 和 3D Gaussian Splatting 在机器人里的用途 (重建、仿真资产、表征)?
4. 如何把空间/几何信息注入到 VLA 大模型里?
5. ◆ 仅靠 RGB 单目能否做好操作任务? 谈深度信息的必要性。

M9 · 规划、控制与系统

1. ★ 分层架构 (high-level 任务规划 + low-level 控制) 的优劣? 端到端 vs 分层之争。
2. ★ LLM 做任务规划 (task planning) 的典型范式? (SayCan、Code as Policies、ReAct)
3. SayCan 的 affordance grounding 思想是什么? 解决了 LLM 的什么问题?
4. ★ 长程任务 (long-horizon) 为什么难? 如何做子目标分解?
5. 控制频率、感知频率、规划频率不一致时系统如何协调?
6. ◆ 端到端 VLA 和“LLM 规划 + 技能库”两条路线, 你赌哪个能走向通用? 为什么?

M10 · 经典论文与 SOTA (要能讲清贡献与局限)

1. ★ RT-1 $\rightarrow$ RT-2 $\rightarrow$ RT-X 的演进脉络, 每一步解决了什么?
2. ★ OpenVLA: 架构、训练数据、关键结论。
3. ★ $\pi_0 / \pi_{0.5}$ (Physical Intelligence): flow matching action expert 的设计。
4. RDT-1B (清华): 扩散 + 大模型, 双臂操作的贡献?
5. Octo: 通用机器人策略 transformer 的设计取舍?
6. ★ Gemini Robotics / GR00T (NVIDIA) / Helix (Figure) 你了解多少? 各自卖点?
7. ★ Diffusion Policy、ACT 这两篇”操作策略经典”必须能默写架构。
8. ◆ 选一篇你最近读过的具身论文, 讲清 motivation、方法、实验、你的批判。

M11 · 工程与部署

1. ★ 大模型 VLA 推理慢, 部署到机器人上有哪些加速手段? (量化、蒸馏、缓存、异步)
2. action chunking / receding horizon 如何在工程上平衡延迟与平滑?
3. ★ 边缘设备 (Jetson 等) 算力受限, 如何做模型裁剪与调度?
4. 实时控制对延迟和抖动 (jitter) 的要求? 大模型如何不阻塞控制回路?
5. ◆ 你会如何搭一套从数据采集 $\rightarrow$ 训练 $\rightarrow$ 评测 $\rightarrow$ 部署的完整 MLOps 闭环?

M12 · 开放题与项目面

1. ★ 介绍一个你做过的具身/机器人/大模型项目：问题、方法、你的贡献、结果、复盘。
2. ★ 你这个方法相比 baseline 好在哪？如果再给你两周你会怎么改进？
3. ★ 你怎么看具身智能未来 3 年的技术路线？哪些是泡沫，哪些是真趋势？
4. 数据 vs 算法 vs 算力，你认为当前具身的瓶颈主要在哪？
5. ◆ Scaling law 在具身上成立吗？数据规模上去性能就一定上去吗？
6. ◆ 如果让你从零带队做一个家庭服务机器人的” 大脑”，技术路线怎么定？
7. (反问环节) 你会问面试官/团队哪些问题来判断这个岗位值不值得去？

学习路径建议

- 第 1 轮 (建立框架)：M1 → M2 → M3，先把” 具身 + 大模型” 的主干打通。
- 第 2 轮 (策略核心)：M4 → M5 → M6，吃透策略学习三大流派。
- 第 3 轮 (数据与系统)：M7 → M8 → M9，理解落地工程。
- 第 4 轮 (冲刺)：M10 论文精读 + M11 工程 + M12 模拟面。

进度记录见同目录 99-进度追踪.md (可选)。

2 M1 • 具身智能基础概念—逐题精讲

目标：把”具身 + 大模型”的底层叙事讲清楚，面试一开口就有框架感。

Q1 ★ 什么是具身智能？与传统机器人控制、纯语言大模型的本质区别？

一句话定义

具身智能（Embodied AI）= 智能体**拥有物理身体**，通过**感知-行动闭环**与**真实（或仿真）环境**持续交互、并在交互中学习的智能。智能不是”被灌进去的知识”，而是在**身体与环境耦合中涌现**出来的。

三方对比（面试可直接背这张表）

维度	传统机器人控制	纯语言大模型 (LLM)	具身智能大模型
智能来源	人工建模 + 控制律	海量文本统计规律	多模态交互数据中学习
输入	结构化传感器	文本/图像	视觉 + 语言 + 本体感知 (proprioception)
输出	预设控制指令	文本 token	物理动作 (连续/高维/高频)
环境耦合	有，但靠人工建模	无（开环、无身体）	有，闭环、可试错
泛化	弱，换场景重调	语义泛化强	目标：语义泛化 + 物理泛化
失败代价	实时安全关键	仅输出错误	可能损坏硬件/伤人 ，不可回退

本质区分的三个要点

- 输出空间不同**：LLM 输出离散 token，机器人要输出**连续、高维、带时序、有物理约束**的动作。这是 VLA 最核心的难点。
- 闭环 vs 开环**：LLM 是”问一句答一句”的开环；具身是”动作改变世界、世界又反馈”的闭环，误差会**累积** (compounding error)。
- 数据性质不同**：文本是”免费”的互联网数据；具身数据要靠遥操作/仿真**昂贵采集**，且存在 Sim2Real gap。

加分句：”纯 LLM 是 *Internet AI*，知道’世界是什么样’；具身智能要回答’我该怎么动’，需要把语义 *grounding* 到物理动作上。”

Q2 ★ ”具身”为什么重要？从感知-行动闭环解释

- **认知科学依据**：具身认知（Embodied Cognition）认为，智能离不开身体——概念的意义部分来自身体经验（如”重””上下”源于身体感受）。
- **闭环的价值**：感知 → 决策 → 行动 → 改变环境 → 新感知。机器人通过”动”来主动获取信息（active perception，如换个角度看遮挡物），并即时纠错。
- **对比开环**：开环系统一步错步步错；闭环可以根据反馈实时修正，是应对真实世界**不确定性**的根本机制。

Q3 摩拉维克悖论（Moravec’s Paradox）

内容：对人类很难的事（下棋、证明定理）对机器很容易；对人类很容易的事（抓取、行走、感知）对机器极难。

原因：高级推理是近期演化、可符号化的；而感知运动能力是亿万年演化打磨、高度并行、难以形式化的”隐性智能”。

对具身的意味：大模型解决了”高级认知”（语言、推理）后，**真正的硬骨头恰恰是底层的感知-操作-运动控制**——这正是具身智能要啃的部分。这也是为什么”会聊天的 AI”早就有了，”会叠衣服的机器人”还很难。

Q4 符号主义 / 联结主义 / 具身认知三条路线

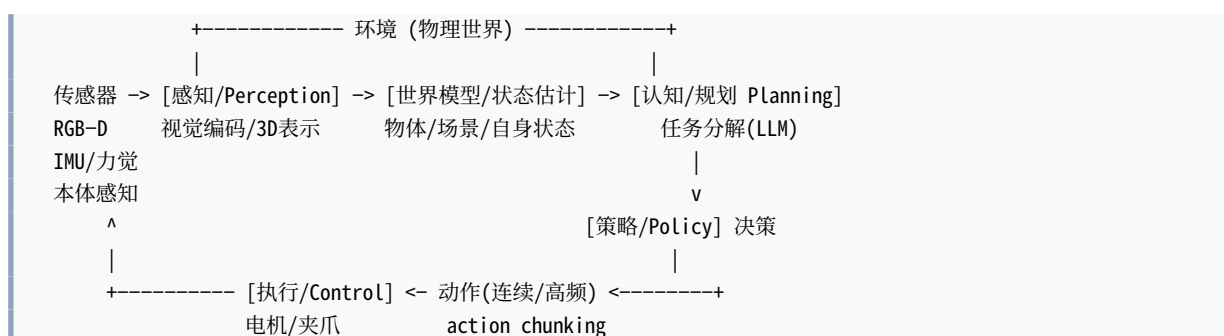
路线	主张	代表	局限
符号主义 (GOF AI)	智能 = 符号操作 + 逻辑推理	专家系统、规划	脆弱、难处理感知
联结主义	智能 = 神经网络分布式表示	深度学习、大模型	黑箱、需大数据
具身认知	智能根植于身体与环境交互	Brooks 的行为机器人	难规模化、缺高层认知

具身智能大模型本质是**联结主义（大模型）+ 具身认知（身体闭环）**的融合，并常借符号主义做高层任务规划（如 LLM planner）。

Q5 ★ 大模型时代，具身相比”互联网 AI”的独特挑战

1. **数据稀缺且昂贵**：没有”机器人版互联网”，遥操作一条轨迹成本高。
2. **实时性**：控制回路要几十 ~ 上千 Hz，大模型推理慢，存在频率鸿沟。
3. **安全不可逆**：物理世界试错可能损坏硬件或伤人，不能像聊天那样”重来”。
4. **长尾与泛化**：真实世界物体/场景/光照无穷组合，分布外（OOD）样本是常态。
5. **本体异构**：不同机器人自由度/形态不同（cross-embodiment），策略难直接迁移。
6. **评测难**：真机评测慢、贵、难复现，sim 与真机成功率常不一致。

Q6 一个完整具身智能体的 pipeline



- **Sense**: 多模态传感 → 编码 (ViT/点云网络)。
- **Think**: 状态估计、世界模型预测、LLM 高层规划、策略输出动作。
- **Act**: 低层控制器 (PD/MPC/力控) 执行, 结果反馈回感知, 闭环。

端到端 VLA 把 "Think + 部分 Act" 压进一个大模型; 分层架构则显式拆分。两条路线见 M9。

Q7 ♦ 你认为"通用机器人脑"最大的瓶颈是什么? (开放题示范答法)

没有标准答案, 关键是有框架、有取舍、有自己的判断。一个稳妥的结构化回答:

"我认为当前主瓶颈是数据, 其次是评测与泛化的闭环。- 数据: 大模型在 NLP/CV 成功靠互联网级数据, 但机器人没有等价数据源, 遥操作成本高、覆盖窄, 这直接卡住了 *scaling*。- 但我不认为只堆数据就够: 还需要 *Sim2Real*、*cross-embodiment* 来放大数据价值, 需要更好的动作表示 (*diffusion/flow*) 来吃下多模态数据, 需要可靠真机评测来形成 *data flywheel*。- 算法和算力相对不是首要瓶颈——架构上 VLA 已收敛, 算力可堆。所以我会赌'数据 + 评测闭环'是未来两年的主战场。"

这样回答展示了: 知道 *scaling law*、知道数据/算法/算力三要素、有自己的赌注且能自治。

本模块自测 (合上文档能答出来吗)

自测清单

- ☐ 用一句话区分 LLM 和 VLA 的输出空间
- ☐ 摩拉维克悖论 + 对具身的含义
- ☐ 具身相比互联网 AI 的 3 个独特挑战
- ☐ 画出 sense-think-act 闭环
- ☐ 对"最大瓶颈"给出有取舍的开放回答

3 M2 · 多模态大模型基础（VLM）—逐题精讲

目标：讲清”视觉怎么进大模型、怎么对齐”。VLM 是 VLA 的地基，很多 VLA 直接拿预训练 VLM 当 *backbone*。

Q8 ★ 视觉信息如何输入大模型？四种主流方式对比

核心问题：图像是 2D 像素，LLM 吃的是 1D token 序列，如何转换并接入？

方式	代表	机制	优缺点
ViT patch → token	ViT, LLaVA	图像切成 16×16 patch, 线性投影成 token, 直接拼到文本 token 前	简单有效; token 多、算力大
对比预训练编码器	CLIP ViT	用 CLIP 视觉编码器出全局/patch 特征, 再投影	语义对齐好; 分辨率受限
Q-Former (查询压缩)	BLIP-2	一组可学习 query 通过 cross-attention 从图像特征”抽取”固定数量 token	大幅压缩 token 数; 训练复杂
Cross-attention 注入	Flamingo	在 LLM 层间插入 gated cross-attention, 图像作为 KV	不改 LLM 主干; 改动侵入式

记忆主线：LLaVA= 最简单的”投影拼接”；BLIP-2=Q-Former 压缩；Flamingo= 层间 cross-attn。
面试常让你对比 LLaVA vs BLIP-2。

LLaVA 路线 (*visual tokens* 直接拼接 + 一个 *MLP projector*) 因为简单、可扩展，成了当下 VLM/VLA 的主流。

Q9 ★ CLIP 的训练目标？对比学习为什么对具身有用？

- 目标：图文对比学习。一个 batch 内 N 个图文对，构成 $N \times N$ 相似度矩阵，**对角线（匹配对）拉近、非对角线推远**（InfoNCE 损失）。
- 产物：一个把图像和文本映射到**同一语义空间**的编码器——零样本分类、检索都靠它。
- 对具身的价值：
 - 提供**语义对齐的视觉特征**，让机器人能把”红色杯子”这种语言指令 grounding 到图像区域。
 - 海量图文预训练带来**开放词汇泛化**，遇到没见过的物体也有语义先验。
 - 很多 VLA 的视觉 backbone 就是 CLIP/SigLIP，省去从零学视觉。

Q10 主流 VLM 对齐方式速记

模型	对齐方式	一句话
LLaVA	MLP projector 把 CLIP 特征投到 LLM 词空间, visual token 直接拼接	简单粗暴, 指令微调
BLIP-2	Q-Former 两阶段 (表示学习 + 生成学习)桥接冻结的视觉编码器和 LLM	参数高效
Flamingo	Perceiver Resampler + gated cross-attention 插入 LLM	少样本上下文学习强
Qwen-VL	ViT + 适配层, 支持多分辨率/定位	工业级、中文友好

Q11 什么是 visual token? token 太多怎么办?

- **visual token**: 图像经编码器/切 patch 后得到的、能喂进 transformer 的特征向量序列。一张高分图可能产生上千 token。
- **问题**: transformer 自注意力是 $O(n^2)$, token 多 $\rightarrow$ 显存爆、推理慢; 对实时具身尤其致命。
- **压缩手段**:
 - Q-Former / Perceiver Resampler (固定数量 query 抽取)
 - token merging / pruning (合并相似 token)
 - 降分辨率 / 自适应分辨率 (AnyRes)
 - 池化 (pooling)

对具身而言, token 压缩 = 推理提速 = 控制频率提升, 是工程刚需。

Q12 ★ 为什么 VLA 常直接复用预训练 VLM?

复用 VLM backbone 带来三类先验:

1. **语义先验**: 理解” 把抽屉里的苹果拿出来” 的指令语义、物体类别、空间关系。
2. **视觉常识**: 识别物体、材质、遮挡, 开放词汇泛化。
3. **世界知识**: 知道” 刀很锋利”” 杯子装水”, 利于安全和合理操作。

这让 VLA 不必从零学感知和语言, 只需学” 语义 $\rightarrow$ 动作” 的映射——**数据效率大幅提升**, 也是 RT-2/OpenVLA 的核心思路。

Q13 模态鸿沟 (modality gap)

- **定义**: CLIP 等对比学习模型中, 图像 embedding 和文本 embedding 各自聚成两簇, 在共享空间里**存在系统性间隔**, 并非完全混合。

- **成因**：对比损失只要求”匹配对相对更近”，不要求两模态分布重合；加上初始化和温度系数的影响。
- **影响**：跨模态检索/生成时可能有偏差；对需要精细 grounding 的具身任务，需注意特征是否真对齐。

Q14 ♦ 视频理解相比图像，对具身的价值与难点

- **价值**：
 - 提供**时序/动态**信息（运动、因果、物体如何被操作）。
 - 人类操作视频是**海量、低成本**的具身先验来源（学”怎么做”）。
 - 利于学习**世界模型**（预测未来帧）。
- **难点**：
 - token 数随帧数暴涨，算力/显存压力大。
 - 缺动作标签（视频里没有机器人关节角），存在 embodiment gap，要做动作推断或表征学习。
 - 视角、人手 vs 机械手的形态差异。

前沿思路：从人类视频学可迁移表征/*affordance*/奖励，再用少量机器人数据对齐动作。

本模块自测

自测清单

- ☐ LLaVA / BLIP-2 / Flamingo 三种视觉接入方式各一句话
- ☐ CLIP 训练目标 + 对具身的 3 点价值
- ☐ visual token 过多的问题与 2 种压缩法
- ☐ VLA 复用 VLM 带来的 3 类先验
- ☐ 视频对具身的价值与 2 个难点

4 M3 • VLA 模型 (Vision-Language-Action) — 逐题精讲

整个面试的核心模块。VLA = 看 (Vision) + 懂 (Language) + 动 (Action)，把多模态大模型的语义能力接到机器人动作上。必须能默写架构、讲清“动作怎么表示”的三条路线。

Q15 ★ 什么是 VLA 模型？输入输出与典型架构

- 定义：Vision-Language-Action 模型，输入图像观测 + 语言指令 (+ 本体状态)，输出机器人动作的端到端大模型。
- 输入：
 - 视觉：一或多路 RGB(-D) 相机
 - 语言：自然语言指令 (“把可乐递给我”)
 - 本体感知 (proprioception)：关节角、夹爪状态 (可选)
- 输出：动作，通常是末端执行器位姿增量 $\Delta(x,y,z,roll,pitch,yaw)$ + 夹爪开合，或关节空间动作。
- 典型架构：

```
图像 - 视觉编码器(ViT/CLIP) --+
                                +- 大模型 Transformer - Action Head - 动作
指令 - 文本 tokenizer -----+      (三种路线见Q16)
本体状态 - (投影) -----+
```

本质：复用预训练 VLM 的语义理解 + 一个动作输出模块，用机器人数据微调“语义 → 动作”映射。

Q16 ★★ 机器人动作如何被模型表示？三条路线（最核心）

动作的难点：连续、高维、高频、多模态分布。三种主流表示：

路线 A：动作离散化为 token (autoregressive)

- 做法：把每个动作维度分箱 (binning) 成离散 token (如 256 个 bin)，像生成文字一样自回归预测动作 token。
- 代表：RT-1, RT-2, OpenVLA。
- 优点：完美复用 LLM 架构和词表，能直接接 VLM。
- 缺点：离散化有量化误差；自回归逐维生成慢；难表达精细连续控制。

路线 B：直接回归连续值 (regression)

- 做法：MLP head 直接输出连续动作向量，用 MSE/L1 监督。

- 优点：简单、快。
- 缺点：MSE 回归会对多模态动作“取平均”（见 Q27），遇到“可以左绕也可以右绕”时输出中间的错误动作。

路线 C：生成式 head (diffusion / flow matching)

- 做法：用扩散模型或 flow matching 把动作当作分布来采样生成。
- 代表：Diffusion Policy, π_0 (flow matching), RDT。
- 优点：天然建模多模态动作分布；动作平滑、精度高。
- 缺点：采样需多步迭代 (diffusion)，推理较慢 (flow matching 改善了这点)。

面试金句：“动作离散化让我们能复用 LLM，但牺牲精度和速度；diffusion/flow 能建模多模态分布、精度高，是当前 SOTA 操作策略的主流。”

Q17 ★ RT-2 的核心思想

- 一句话：把机器人动作编码成 token，当成一种“外语”，让预训练的视觉-语言模型 (PaLI-X/PaLM-E) 像输出文本一样输出动作 token，从而把互联网级的视觉语言知识迁移到机器人控制。
- 关键做法：
- 动作离散成 256 个 bin，占用文本 token 词表中的一些 token id。
- co-fine-tuning：同时用网络图文数据 + 机器人数据训练，保住语义泛化。
- 重要结论：涌现出语义泛化能力——能听懂“把草莓放进正确的碗”“拿起灭绝动物的玩具（恐龙）”这类需要常识推理的新指令。
- 局限：闭源、巨大、推理慢、高频控制难。

Q18 ★ OpenVLA 相比 RT-2 的改进 / 为何能开源复现

- 背景：RT-2 闭源、超大。OpenVLA (7B) 做了开源、可复现版本。
- 关键设计：
- Backbone：Prismatic VLM (SigLIP + DINOv2 双视觉编码器 + Llama2-7B)，双编码器兼顾语义和空间细节。
- 动作仍走离散化 token 自回归。
- 在 Open X-Embodiment (97 万 + 轨迹) 上训练，跨本体。
- 结论：7B 模型超过 55B 的 RT-2-X；支持 LoRA 高效微调和量化部署。
- 意义：把 VLA 平民化，成了学术界最常用的 VLA baseline。

Q19 ★ action chunking 是什么？为什么有用

- 定义：一次预测未来一段连续动作序列（一个 chunk，如未来 H 步），而不是只预测下一步。
 - 为什么有用：
 - 缓解误差累积：减少决策次数 → 减少 compounding error。
 - 减少抖动、动作更平滑：连贯输出一段轨迹。
 - 应对非马尔可夫/时序依赖，处理人类示范中的停顿和多模态。
 - 降低对大模型的调用频率：一次推理管 H 步，缓解”大模型慢 vs 控制要快”的矛盾。
 - 配套技术：temporal ensemble（ACT 提出）——对重叠时间步的多次预测做加权平均，进一步平滑。
 - 出处：ACT 论文提出，现已成 VLA/Diffusion Policy 标配。
-

Q20 ★ VLA 动作频率受限于什么？慢推理与高频控制如何协调

- 瓶颈：大模型一次前向几十 ~ 几百 ms，输出频率可能只有几 ~ 几十 Hz，而底层力控/电机要几百 ~ 1kHz。
 - 协调手段：
 - action chunking：一次出一段，低频规划 + 高频回放。
 - 分层（hierarchical）：大模型出子目标/中层动作（低频），低层控制器（MPC/PD）做高频跟踪。 $\pi 0.5$ 、Helix 都是这种 system1/system2 思路。
 - 异步执行：大模型异步推理，控制器持续运行不阻塞。
 - 推理加速：量化、蒸馏、flow matching 减少采样步数。
-

Q21 ★ $\pi 0$ (Pi-Zero) 用 flow matching 输出动作的优势

- 背景：Physical Intelligence 的 $\pi 0$ 。VLM backbone（PaliGemma）+ 独立的 action expert，用 flow matching 生成连续动作 chunk。
 - flow matching vs 离散 token：
 - 输出连续高频动作（50Hz），适合灵巧操作（叠衣服、打包）。
 - 比 diffusion 采样步数少、推理更快。
 - 天然建模多模态分布，动作平滑精确。
 - vs 回归：避免 MSE 取平均问题。
 - 是”VLM 语义 + 生成式连续动作”的代表作。
-

Q22 ◆ 如何设计 VLA 实现 zero-shot 泛化（开放题）

结构化答法，从三个层面：

1. **复用强 VLM backbone**：继承开放词汇语义和视觉常识 → 指令/物体语义泛化。
2. **数据多样性**：跨本体（Open X-Embodiment）、多场景、多任务训练 + co-fine-tuning 保住网络知识。
3. **动作表示与抽象**：用相对位姿/物体中心表示降低对绝对坐标的依赖；用 affordance / 语言子目标做中间抽象，提升组合泛化。
4. **(加分)**：引入人类视频学可迁移表征；用 chain-of-thought / 推理增强长程组合泛化（如 RT-H、ECoT）。

本模块自测（重点中的重点）**自测清单**

- ☐ 画出 VLA 架构图，说清三种输入
- ☐ 动作表示三条路线 + 各自优缺点（最高频）
- ☐ RT-2 一句话核心 + co-fine-tuning 作用
- ☐ OpenVLA 的双视觉编码器 + 数据集 + 结论
- ☐ action chunking 的 4 个好处
- ☐ 慢推理 vs 高频控制的 4 种协调方式
- ☐ flow matching 相比离散 token / 回归的优势

5 M4 · 模仿学习与策略学习—逐题精讲

操作策略的核心流派。*Diffusion Policy* 和 *ACT* 是必须能默写架构的两篇经典，和 *M3* 的“动作表示路线 *C*”直接相连。

Q23 ★ 行为克隆 (BC) 原理与最大问题

- 原理：把策略学习当监督学习。收集专家 (观测 $o \rightarrow$ 动作 a) 数据对，训练 $\pi(a|o)$ 拟合专家动作。简单直接。
- 最大问题——分布偏移 (distribution shift) / 复合误差 (compounding error):
- 训练时只见过专家访问过的状态分布；
- 推理时策略一旦有小误差，就进入没见过的状态，误差进一步放大，状态越偏越远——误差随时间步二次方累积 ($O(\epsilon T^2)$ ，DAgger 论文的经典结论)。
- 直观：学开车只看老司机正常行驶，从没见过”车快冲出路面怎么打方向”，一旦偏出就不会救。

Q24 ★ DAgger 如何缓解分布偏移？代价

- DAgger (Dataset Aggregation): 迭代式。
- 用当前策略自己去跑，收集它实际访问到的状态；
- 让专家对这些状态标注正确动作；
- 把新数据聚合进数据集，重训；循环。
- 效果：让训练分布覆盖策略自己会遇到的状态，把误差从 $O(\epsilon T^2)$ 降到 $O(\epsilon T)$ 。
- 代价：需要专家在线持续标注（昂贵，机器人上专家 = 人遥操作纠正），交互成本高，实际部署受限。

Q25 ★★ Diffusion Policy 核心思想（必考）

- 一句话：把”给定观测预测动作”建模为条件去噪扩散过程——从高斯噪声出发，以观测为条件，逐步去噪生成一段动作序列 (action sequence)。
- 为什么用扩散建模动作比直接回归好：
- 天然建模多模态动作分布：同一观测下可能有多种合理动作（左绕/右绕），扩散能表达多峰分布并采样其一；回归会取平均得到错误动作。
- 高维动作空间表达力强，适合 action chunking（一次出一段）。
- 训练稳定（去噪目标），优于 GAN/能量模型。
- 关键设计：
- receding horizon control: 预测 H 步，只执行前几步，再重新预测（滚动）。

- 条件方式：观测作为条件注入 (FiLM / cross-attn)，可用 CNN(UNet) 或 Transformer 去噪网络。
- 输出动作序列而非单步 (action chunking 思想)。
- 代价：采样需多步迭代去噪，推理偏慢 (可用 DDIM/一致性模型/flow 加速)。

Q26 ★ ACT (Action Chunking with Transformers) 关键设计

- 背景：低成本双臂遥操作平台 ALOHA 的配套算法，做精细操作。
- 两个关键设计：
 - **Action Chunking**：一次预测未来 k 步动作序列，降低有效决策频率 → 缓解复合误差；推理时 **temporal ensemble** (对重叠步加权平均) 让动作平滑。
 - **CVAE (条件变分自编码器)**：
 - 训练时 encoder 把”动作序列 + 观测”编码成隐变量 z (捕捉人类示范的**多模态/风格变化**)；
 - decoder (Transformer) 根据观测和 z 重建动作序列；
 - 推理时 z 取先验均值 (0)，只用 decoder。
 - **CVAE 的作用**：建模人类示范的多模态性，避免回归取平均，让策略更稳。
 - 整体是 Transformer encoder-decoder 架构。

Q27 ★ 多模态动作分布 / 为什么 MSE 回归会”取平均”失败

- **多模态动作分布**：同一状态下存在**多个都正确但互斥**的动作 (绕过障碍可左可右；抓杯子可从上可从侧)。其概率分布是**多峰的**。
- **MSE 回归的失败**：MSE 的最优解是条件期望 $E[a|o]$ ，即对所有正确动作**取平均**。两个峰(左/右)的平均落在中间——一个**两边都不对**的动作 (直接撞上障碍)。
- **解法**：用能建模多峰分布的方法——**Diffusion Policy**、**CVAE(ACT)**、**flow matching**、**离散化 + 分类**、**混合密度网络 (MDN)**。
- 这是理解”为什么具身策略要用生成式模型”的关键，常被追问。

Q28 ★ 如何评估模仿策略？为何 sim 与真机成功率不一致

- **评估指标**：任务**成功率** (success rate)、完成时间、鲁棒性 (扰动/新物体下的表现)、平滑度。
- **sim $\neq$ real 的原因**：
 - 动力学/接触力建模不准 (接触、摩擦最难仿真)。
 - 渲染与真实图像差异 (视觉域偏移)。
 - 传感器噪声、延迟、标定误差在真机才有。
 - sim 里可”重置”，真机有累积漂移。

- 启示：真机评测才是金标准，但慢且贵；要建立可靠的真机评测流程，避免被 sim 数字误导。

Q29 ♦ 只有少量示范，如何提升鲁棒性（开放题）

- 数据层：数据增强（视觉随机化、随机裁剪/颜色）、加入次优/恢复数据教”如何纠错”。
- 算法层：用预训练表征（VLM/视觉基础模型）降低样本需求；action chunking 减少复合误差；diffusion/CVAE 建模多模态。
- 范式层：DAgger 式在线纠正补关键状态；从人类视频做表征预训练；少量真机 + 大量仿真协同。
- 结构层：物体中心/相对位姿表示提升泛化；引入语言子目标做组合泛化。

本模块自测

自测清单

- ☐ BC 的复合误差为什么是 $O(\epsilon T^2)$ ，DAgger 怎么降到 $O(\epsilon T)$
- ☐ Diffusion Policy 一句话 + 为何优于回归 + receding horizon
- ☐ ACT 的两个关键设计（chunking + CVAE）及 CVAE 作用
- ☐ 用”绕障碍左右”例子讲清 MSE 取平均失败
- ☐ sim 与真机成功率不一致的 3 个原因

6 M5 · 强化学习在具身—逐题精讲

RL 在具身里更多用于：运动控制 (*locomotion*)、*sim* 中训练、以及策略微调。要讲清和模仿学习的取舍。

Q30 ★ RL vs IL 在具身中的优劣与选型

维度	模仿学习 (IL)	强化学习 (RL)
数据	需专家示范	需奖励函数 + 大量交互
上限	受限专家水平	可超越专家 (探索)
样本效率	高 (有示范)	低 (试错多)
安全	较安全 (模仿好行为)	探索有风险
主要难点	分布偏移、示范成本	奖励设计、探索、sim2real

- **何时用 IL**：有高质量示范、任务难定义奖励 (如灵巧操作、长程任务)、真机直接学。
- **何时用 RL**：奖励易定义、可大规模仿真试错 (如四足/人形 *locomotion*)、需要超越示范或精细优化。
- **现实主流**：IL 打底 + RL 微调。先用示范 BC/Diffusion 学个能用的策略，再用 RL 在仿真里精修 (如 RLHF 式偏好优化、residual RL)。

Q31 on-policy(PPO) vs off-policy(SAC) 的取舍

- **PPO (on-policy)**：用当前策略采的数据更新，稳定、易调，但**样本效率低** (数据用一次就丢)。仿真里并行环境多、数据廉价时首选——四足/人形 RL 几乎都用 PPO。
- **SAC (off-policy)**：用 replay buffer 复用历史数据，**样本效率高**，适合真机/采样昂贵场景；但调参敏感、易不稳定。
- **选型主线**：仿真海量并行 → PPO；真机样本贵 → SAC/off-policy。

Q32 ★ 稀疏奖励问题怎么解决

稀疏奖励：只在任务完成才给 1，过程全 0，智能体长时间收不到信号、难探索。

- **Reward shaping**：人为加中间奖励 (距离目标越近奖励越高)。注意：塑形不当会引入**奖励黑客** (reward hacking)。
- **HER (Hindsight Experience Replay)**：把失败轨迹的“实际到达状态”当作目标重新标注，让失败也有学习信号——稀疏奖励神器。
- **课程学习 (curriculum)**：从易到难逐步增加任务难度。
- **示范引导**：用少量 demo 初始化或加 BC 损失 (如 DDPGfD、AWAC)。
- **内在奖励 (curiosity/exploration bonus)**：鼓励探索新状态。

Q33 ★ 真机 RL 为什么难? sim 训练迁移的障碍

- **真机 RL 难**: 试错可能损坏硬件/危险; 采样慢 (真实时间不能加速); 需要自动 reset; 奖励难自动测量。
- **sim→real 障碍**: Sim2Real gap——动力学差异、渲染差异、传感器噪声、延迟。解法见 M7 (域随机化、系统辨识、real-to-sim)。

Q34 RLHF 思想能否迁移到具身

- 可以。思路: 用人类偏好/反馈学习奖励模型, 再优化策略。
- 形式:
- **偏好学习**: 人对两条机器人轨迹比较好坏 → 训练奖励模型 → RL 优化 (如 PEBBLE、RLHF for robotics)。
- **语言反馈**: 人用语言纠正 (“再往左一点”), 转成奖励或目标修正。
- 价值: 解决具身**奖励难手工设计**的痛点, 尤其对”什么算做得好”主观的任务。

Q35 ♦ ”把杯子放到架子上”奖励设计 (开放题示范)

结构化拆解 (展示你懂 reward shaping 的坑):

- **稀疏主奖励**: 杯子最终稳定在架子上 +1 (定义成功的客观判据)。
- **塑形项** (加速学习):
- 接近奖励: 夹爪 → 杯子距离、杯子 → 目标距离递减。
- 抓取奖励: 成功夹住 + 小奖励。
- 姿态奖励: 杯子保持竖直 (不洒)。
- **惩罚项**: 碰撞、掉落、动作过大/能耗、超时。
- **会踩的坑**:
- **reward hacking**: 比如奖励”杯子靠近架子”, 策略可能把杯子扔过去而非放好 → 需加稳定性/速度约束。
- 塑形权重失衡导致只优化某一项。
- 距离奖励的局部最优 (卡在次优姿态)。
- **更稳的做法**: 尽量用稀疏成功奖励 + 少量 demo + HER, 减少手工塑形带来的偏差。

本模块自测

自测清单

- ☐ RL vs IL 选型，以及“IL 打底 +RL 微调”主流范式
- ☐ PPO vs SAC：仿真用哪个、真机用哪个，为什么
- ☐ 稀疏奖励 4 种解法（重点 HER 思想）
- ☐ 真机 RL 难的 3 个原因
- ☐ 给一个任务能口头设计奖励并指出 reward hacking 风险

7 M6 · 世界模型与预测—逐题精讲

前沿加分模块。世界模型让智能体“在脑中想象”，是 *model-based* 方法和近期“视频生成即策略”路线的核心。

Q36 ★ 什么是世界模型 (World Model)? 在具身中的角色

- **定义**：一个能预测环境动态的模型——给定当前状态和动作，预测下一个状态（和奖励）。即学习环境的“内部模拟器” $P(s'|s,a)$ 。
- **角色**：
 - **想象中规划/训练**：在世界模型里“做梦”试错，不消耗真实交互（样本效率高、安全）。
 - **预测性决策**：MPC 式地展开多步预测，选最优动作。
 - **表征学习**：预测任务逼模型学到有用的环境结构表征。
 - **经典**：Ha & Schmidhuber 的“World Models”；Dreamer 系列；近期用视频生成模型当世界模型。

Q37 基于模型 RL (MBRL) vs model-free 的优劣

- **MBRL 优点**：
 - **样本效率高**：在学到的模型里生成大量虚拟交互。
 - **可做规划、可迁移到新任务**（模型与奖励解耦）。
- **MBRL 风险**：
 - **模型误差累积**：在不准的模型里展开多步，误差放大，策略可能利用模型缺陷（model exploitation）。
 - **模型学习本身有成本**。
- **model-free**：直接学策略/价值，渐近性能好、实现简单，但样本效率低。
- **取舍**：样本昂贵（真机）→ 倾向 MBRL；仿真廉价 → model-free 也可。

Q38 ★ 用视频生成模型做世界模型/策略

- **思路**：视频生成模型（Sora 类、扩散视频模型）本质学了**视觉动态规律**，可作为世界模型：
- 给定当前帧 + 动作/语言 → 预测未来帧 → 据此规划（visual planning）。
- **代表工作**：UniPi（“文生视频当策略”）、UniSim、Genie、各类 interactive world model。
- **局限**：
 - 生成慢、算力大，难满足实时控制。

- 像素级预测未必抓住物理/接触关键，可能“看起来对、物理错”。
- 缺动作标签时需 inverse dynamics 把视频转成动作。
- 长程一致性、幻觉问题。

Q39 Dreamer 系列核心机制 (latent imagination)

- **核心**：在隐空间 (latent space) 里学世界模型并“想象”训练，而非在像素空间。
- **关键组件 (RSSM, Recurrent State-Space Model)**：把高维观测编码成紧凑隐状态，在隐空间预测未来。
- **latent imagination**：策略和价值完全在想象的隐轨迹上训练（不需真实环境多次交互）→ 样本效率极高。
- DreamerV3 在多种任务（含从零采集钻石的 Minecraft）上用同一套超参成功，展示通用性。

Q40 ♦ ”预测下一帧” vs ”预测下一个动作” 哪个更本质（开放题）

展示思辨，给有取舍的回答：

”两者是互补的，对应**理解世界**和**做出行为**两个层面。- 预测下一帧（世界模型）逼模型理解**环境因果与动态**，提供可迁移的通用表征、支持规划和想象训练，更接近‘理解世界’。- 预测下一个动作（策略）直接服务于**控制目标**，是任务落地必须的。- 我的看法：**预测帧更基础、更通用，但不充分**——它学到的是 *observational* 规律，不含‘我该如何介入’的因果干预信息。理想架构是世界模型提供表征与想象，策略在其上做决策，两者联合训练（如 *Dreamer*）。当前纯视频世界模型的瓶颈正是缺乏动作 *grounding* 和物理保真。”

本模块自测

自测清单

- ☐ 世界模型定义 + 在具身的 3 个角色
- ☐ MBRL vs model-free 优劣，model exploitation 风险
- ☐ 视频生成当世界模型的思路 + 2 个局限
- ☐ Dreamer 的 latent imagination 是什么
- ☐ 对”预测帧 vs 预测动作”给出有取舍的回答

8 M7 · 数据、仿真与 Sim2Real 一题精讲

落地最实在的模块。”数据从哪来””Sim2Real 怎么办”几乎是工业岗必问。数据是具身当前最大瓶颈。

Q41 ★ 具身数据从哪来？四种来源对比

来源	方式	优点	缺点
遥操作 (teleoperation)	人用手柄/VR/主从臂操控机器人采集	真机真分布、有动作标签、质量高	贵、慢、人力密集
人类视频	网络/第一视角人类操作视频	海量、廉价、多样	无动作标签、embodiment gap (人手 ≠ 夹爪)
仿真 (simulation)	物理引擎里生成	海量、可标注、安全、可随机化	Sim2Real gap
自主采集/RL	机器人自己探索	可扩展、覆盖策略分布	早期策略差、需奖励、安全
数据增强/生成	用生成模型扩充 (如 DexMimicGen、视频生成)	放大已有数据	可能引入伪影

现实：遥操作打底（高质量）+ 仿真规模化 + 人类视频做预训练表征，多源混合。

Q42 ★ Sim2Real gap 是什么？主要来源

- 定义：在仿真里训练好的策略迁移到真机时性能下降的现象，源于仿真与现实的差异。
- 主要来源：
 - 动力学差异：质量、摩擦、接触、电机响应、延迟难精确建模（接触/摩擦最难）。
 - 视觉/渲染差异：仿真图像与真实相机的纹理、光照、噪声不同（visual gap）。
 - 传感器差异：真机有噪声、标定误差、时延。
 - 执行差异：真实电机有死区、饱和、磨损。

Q43 ★ 域随机化 (Domain Randomization) 原理与维度选择

- 原理：训练时大量随机化仿真参数（视觉 + 物理），让真实世界看起来只是”训练分布里的又一个样本”，迫使策略学到对这些变化鲁棒的不变特征。
- 常随机化的维度：
 - 视觉：纹理、颜色、光照、相机位姿、背景。
 - 物理：质量、摩擦系数、阻尼、外力扰动、控制延迟。
- 维度选择原则：随机化你不确定、且真机可能变化的参数；范围要覆盖真实值但别过宽（过宽会让任务过难、策略保守）。

- **进阶**：自动域随机化（ADR，逐步加宽）、**域适应**（用少量真机数据对齐）、**系统辨识**（测量真机参数缩小随机范围）。

Q44 Open X-Embodiment 的意义 / 跨本体难点

- **意义**：Google 牵头联合多机构，汇集 **22 种机器人本体**、**100 万 + 轨迹** 的统一格式大数据集，催生 RT-X、OpenVLA。证明**跨本体共训能提升单本体性能**（正迁移）。
- **跨本体（cross-embodiment）难点**：
 - 不同机器人自由度、动作空间、相机配置、控制频率都不同。
 - 动作语义不统一（关节空间 vs 末端空间）。
 - 解法：统一动作表示（末端位姿）、本体标识/条件输入、对齐观测空间。

Q45 ★ 数据飞轮（data flywheel）怎么转

闭环：部署 → 采集真实数据（含失败案例）→ 标注/筛选 → 重训模型 → 性能提升 → 更大规模部署 → 采更多数据……

- 关键：部署越多、采的长尾数据越多、模型越强，形成正循环。
- 难点：真机采集慢、标注贵、需自动化筛选有价值数据（主动学习挑信息量大的样本）。
- 这是机器人公司的核心壁垒（类比特斯拉 Autopilot 数据闭环）。

Q46 遥操作数据常见质量问题与清洗

- **常见问题**：操作员手抖/犹豫停顿、失败轨迹、节奏不一致、不同操作员风格差异大、标定漂移、时间戳错位（图像-动作不对齐）。
- **清洗**：剔除失败/超时轨迹、平滑滤波、对齐时间戳、统一坐标系、按成功率/平滑度打分筛选、平衡不同操作员/任务分布。

Q47 ◆ 100 台机器人 + 有限预算，如何设计数据采集（开放题）

结构化：

1. **任务/场景优先级**：先覆盖高价值、高频任务，保证基础分布。
2. **多样性 > 单纯数量**：物体、位置、光照、干扰多样，避免冗余重复轨迹。
3. **主动学习**：优先采集模型当前**失败/不确定的**场景（信息量大）。
4. **难度课程**：先简单建立能力，再逐步采难样本。
5. **质量控制**：标准化遥操作流程、实时质检、自动剔除坏数据。
6. **仿真 + 真机配比**：批量用仿真覆盖，真机补关键长尾和接触丰富任务。
7. **形成飞轮**：边部署边采，自动回流失败案例。

核心思想：用有限真机预算采”模型最需要的数据”，其余靠仿真和增强放大。

本模块自测

自测清单

- ☐ 四种数据来源各自优缺点（尤其人类视频的 embodiment gap）
- ☐ Sim2Real gap 的 3 类来源
- ☐ 域随机化原理 + 如何选随机化维度
- ☐ Open X-Embodiment 的意义 + 跨本体难点
- ☐ 数据飞轮怎么转、为什么是壁垒
- ☐ 有预算约束下的数据采集策略（主动学习 + 多样性）

9 M8 • 感知与 3D 表示—逐题精讲

操作任务高度依赖空间几何。偏感知岗会深挖 3D 表示、抓取、空间信息如何注入大模型。

Q48 ★ 为何常需 3D 表示？各种 3D 表示适用场景

- **为何要 3D**：操作是物理空间里的事——抓取、避障、放置都需要**精确的几何与深度**。纯 2D 有尺度/遮挡歧义，难以精确定位末端到物体的相对位姿。
- **常见 3D 表示**：

表示	特点	适用
深度图 (depth)	RGB-D 直接给, 2.5D	近距离操作、避障
点云 (point cloud)	无序点集, 几何精确	抓取位姿估计、配准
体素 (voxel)	规则网格, 可做 3D 卷积	占用预测、碰撞检测 (稀疏 voxel 省内存)
网格 (mesh)	表面拓扑	仿真资产、碰撞
隐式场 (NeRF/SDF)	连续函数表示	高保真重建
3D Gaussian Splatting	显式高斯点, 渲染快	实时重建、仿真资产、表征

Q49 6-DoF 抓取位姿估计常见做法

- **6-DoF 抓取**：预测夹爪在 3D 空间的位置 (3)+ 姿态 (3)，使其能稳定抓住物体。
- **常见方法**：
- **采样-评估**：在点云上采样大量候选抓取, 用网络打分排序 (如 GraspNet / 6-DOF GraspNet / Contact-GraspNet)。
- **直接回归**：网络直接从点云/RGB-D 预测抓取位姿。
- **end-to-end**：把抓取嵌入策略，直接输出动作 (VLA 路线)。
- **评价**：抓取成功率、覆盖率、对杂乱场景/未知物体的泛化。

Q50 ★ NeRF / 3DGS 在机器人里的用途

- **NeRF (神经辐射场)**：用 MLP 表示场景，新视角合成、高保真重建。机器人用途：场景重建、生成仿真资产、做表征。缺点：训练/渲染慢。
- **3D Gaussian Splatting**：用一堆显式 3D 高斯表示场景，**渲染极快 (实时)**、质量高。机器人用途：
- 实时场景重建与数字孪生 (real2sim)。
- 生成逼真仿真环境缩小 visual gap。

- 作为可微表征支持位姿估计、操作规划。
- 趋势：3DGS 因速度优势在机器人里比 NeRF 更受青睐。

Q51 如何把空间/几何信息注入 VLA 大模型

- **输入点云/深度**：用点云编码器（PointNet++/3D backbone）出 token 拼接进大模型。
- **3D position embedding**：给视觉 token 加 3D 坐标/相机参数编码，让模型有空间感（如多视角融合）。
- **多视角图像**：用多相机 + 几何约束隐式提供 3D。
- **物体中心表示**：先检测/分割物体，用其 3D 位姿作为结构化输入。
- **显式中间表示**：先估计 6-DoF 抓取/affordance map，再交给策略。
- **权衡**：纯 2D VLA（如 OpenVLA）简单但空间精度弱；注入 3D 提升精度但增加复杂度和传感器依赖。

Q52 ♦ 仅 RGB 单目能否做好操作？（开放题）

平衡作答：

”部分可以，但有局限。- **支持单目的理由**：大规模数据 + 强视觉先验下，模型能从 2D 学到隐式深度线索（透视、遮挡、阴影、先验尺寸）；RT-2/OpenVLA 等纯 RGB VLA 已能做不少操作；单目硬件便宜、标定简单。- **局限**：精细操作（插孔、对齐、堆叠）需要毫米级深度，单目有尺度歧义；遮挡和透明/反光物体困难。- **我的判断**：粗操作和语义抓取单目够用；高精度接触丰富任务建议 RGB-D 或多视角，或用 action chunking + 闭环视觉伺服补偿。务实做法是单目为主 + 深度/触觉在关键阶段补充。”

本模块自测

自测清单

- ☐ 为何操作需要 3D；点云/voxel/3DGS 各自适用
- ☐ 6-DoF 抓取的采样-评估范式（GraspNet 思路）
- ☐ 3DGS 相比 NeRF 的优势 + 机器人用途
- ☐ 把 3D 信息注入 VLA 的 3 种方式
- ☐ 对”单目能否做操作”给出有取舍的回答

10 M9 · 规划、控制与系统—逐题精讲

体现”系统观”的模块。端到端 vs 分层之争、LLM 任务规划、长程任务是核心。

Q53 ★ 分层架构 vs 端到端：优劣之争

- 分层 (high-level 规划 + low-level 控制):
- high-level: LLM/任务规划器把指令分解成子任务/技能序列。
- low-level: 技能库/控制器执行每个子任务。
- 优点: 可解释、模块化、易调试、可复用技能、长程任务好处理。
- 缺点: 模块间误差传递、接口僵硬、技能库需人工设计、上限受限于技能。
- 端到端 (VLA 直接观测 → 动作):
- 优点: 联合优化无接口损失、可学到难以手工设计的行为、随数据 scale。
- 缺点: 黑箱难调试、长程任务难、数据需求大、安全难保证。
- 之争核心: 分层泛化靠组合已有技能、落地稳; 端到端泛化靠数据和 scaling、上限高但贵。
- 现状趋势: 混合——大模型出语义子目标 (慢、低频), 底层学习/控制做执行 (快、高频), 即 system2+system1 (π 0.5、Helix、Gemini Robotics 的思路)。

Q54 ★ LLM 做任务规划的典型范式

范式	思想	代表
LLM 直接分解	让 LLM 把任务拆成自然语言子步骤	zero-shot planner
affordance grounding	LLM 提议 + 价值函数判断可行性	SayCan
Code as Policies	LLM 直接生成调用 API/技能的代码	Code as Policies
ReAct	推理 (reason) 与行动 (act) 交替, 可调用工具/感知反馈	ReAct
CoT/子目标推理	显式推理中间步骤再出动作	RT-H, ECoT(具身思维链)

Q55 SayCan 的 affordance grounding

- 解决的问题: LLM 懂语言但不知道当前环境下什么动作可行 (可能规划出”去拿苹果”但周围没苹果, 或机器人够不到)。
- 核心思想: 把决策拆成两部分相乘:
- Say: LLM 给出每个候选技能与指令的语义相关性 $p(\text{技能} | \text{指令})$ 。
- Can: 价值函数/affordance 模型给出每个技能在当前状态成功的概率 $p(\text{成功} | \text{状态}, \text{技能})$ 。

- 选 二者乘积最大的技能执行。
- 意义：把 LLM 的”知识”和机器人的”现实能力”结合——既相关又可行。

Q56 ★ 长程任务为什么难？如何分解

- 难点：
 - 步骤多 → 复合误差累积，一步失败全盘崩。
 - 稀疏奖励/延迟反馈，credit assignment 难。
 - 需要记忆和子目标管理。
- 解法：
 - 分层分解：LLM 把长任务拆成子目标，逐个完成（任务与运动规划 TAMP）。
 - 技能复用：训练可组合的原子技能。
 - 闭环重规划：每步后检查状态、失败则重规划（ReAct/反馈）。
 - 记忆/状态跟踪：维护任务进度。

Q57 控制/感知/规划频率不一致如何协调

- 典型频率：底层力控/电机 500Hz~1kHz，感知 30Hz，大模型规划 1~10Hz。
- 协调机制：
 - 分层异步：各层独立频率运行，高层低频出目标，低层高频跟踪。
 - action chunking：大模型一次出一段动作，低层高频回放。
 - 插值/滤波：把低频目标平滑插值到高频指令。
 - 实时性保证：控制回路放实时线程/RTOS，大模型推理异步不阻塞。

Q58 ◆ 端到端 VLA vs ”LLM 规划 + 技能库”，赌哪个走向通用（开放题）

”短期我赌混合架构，长期看好端到端但分层结构化。- LLM 规划 + 技能库现在更落地：可解释、可复用、长程稳，工业部署友好；但上限受技能库限制，接口僵硬，难处理需要紧耦合感知-动作的灵巧操作。- 端到端 VLA 上限更高：能学到手工难设计的灵巧行为，随数据 scaling；但当前数据不足、长程弱、难调试安全。- 我的判断：会收敛到端到端学习的技能 + 大模型做高层语义规划的分层混合（system1/system2），既享受 scaling 又保留可控性。纯符号规划会被淘汰，纯单体端到端短期难独自搞定长程。关键变量是数据规模——数据够了端到端占比会越来越大。”

本模块自测**自测清单**

- ☐ 分层 vs 端到端优缺点, 混合趋势 (system1/system2)
- ☐ LLM 任务规划 4 种范式 (SayCan/Code as Policies/ReAct/CoT)
- ☐ SayCan 的 Say $\times$ Can 乘积思想
- ☐ 长程任务难点 + 分解/重规划解法
- ☐ 频率不一致的协调 (分层异步 + chunking + 实时线程)
- ☐ 对端到端 vs 分层给出有赌注的回答

11 M10 · 经典论文与 SOTA 一题精讲

面试官常问”最近读了什么论文””讲讲 XX 工作”。每篇要能讲清：*motivation* → 方法 → 关键结论 → 局限。

Q59 ★ RT-1 → RT-2 → RT-X 演进脉络

- **RT-1 (2022, Google)**: Robotics Transformer。
- 做法: Transformer 把图像 (EfficientNet+FiLM)+ 指令 → **离散化动作 token**, 模仿学习训练。
- 数据: 13 万真机轨迹、700+ 任务。
- 贡献: 证明 Transformer + 大规模真机数据能学**多任务**策略, 有一定泛化。解决”能否用一个模型做很多任务”。
- **RT-2 (2023)**: Vision-Language-Action。
- 做法: 把动作 token 接到**预训练 VLM** (PaLI-X/PaLM-E) 上, co-fine-tuning。
- 贡献: 迁移互联网视觉语言知识 → **语义泛化与推理** (懂新概念、常识)。解决”如何获得开放世界泛化”。
- **RT-X / Open X-Embodiment (2023)**:
- 做法: 联合 22 种本体、100 万 + 轨迹共训 RT-1-X / RT-2-X。
- 贡献: 证明**跨本体共训正迁移**。解决”如何扩大数据规模”。
- 主线: **多任务 (RT-1) → 语义泛化 (RT-2) → 跨本体规模化 (RT-X)**。

Q60 ★ OpenVLA

- **Motivation**: RT-2 闭源巨大, 社区需要开源可复现的 VLA。
- **方法**: Prismatic VLM = **SigLIP + DINOv2 双视觉编码器** (语义 + 空间细节) + Llama2-7B; 动作走离散 token 自回归; 在 Open X-Embodiment 97 万轨迹训练。
- **结论**: 7B 超 55B RT-2-X 成功率; 支持 LoRA 高效微调、量化部署。
- **局限**: 离散动作、推理慢、单步预测 (无 chunking 原版)。成了学术最常用 baseline。

Q61 ★ $\pi 0$ / $\pi 0.5$ (Physical Intelligence)

- **$\pi 0$** : VLM(PaliGemma) + **action expert**, 用 **flow matching** 输出连续高频动作 chunk (50Hz)。在大规模多样数据上预训练, 能做叠衣服、打包等灵巧长程任务。卖点: 通用 + 连续动作 + 快。
- **$\pi 0.5$** : 强调**开放世界泛化** (到没见过的家庭环境), 用分层 (高层语义子任务 + 低层 flow 动作) + co-training 异构数据。
- 意义: 代表”VLM 语义 + 生成式连续动作 + 分层”的工业 SOTA 路线。

Q62 RDT-1B (清华)

- **Motivation:** 双臂灵巧操作 + 扩大模型与数据。
- **方法:** **Diffusion** + **Transformer** 的 1.2B 大模型 (当时最大扩散策略), 统一异构机器人动作空间, 多机器人数据预训练 + 少量双臂数据微调。
- **贡献:** 扩散策略 scale 到 10 亿级、双臂协调、强 zero-shot 泛化。

Q63 Octo

- **方法:** 开源通用机器人策略 **Transformer**, 在 Open X-Embodiment 80 万轨迹训练; **模块化设计**——支持灵活接入不同观测 (多相机、本体) 和动作空间, 用 **diffusion action head**。
- **取舍:** 相对轻量、易微调适配新机器人; 偏 generalist policy, 不依赖大 VLM backbone。

Q64 ★ 其他前沿 (卖点速记)

- **Gemini Robotics (DeepMind):** 基于 Gemini 多模态, 强语义推理 + VLA; Gemini Robotics-ER 强空间推理 (embodied reasoning)。
- **GR00T (NVIDIA):** 人形机器人通用基础模型, 配 Isaac 仿真 + 合成数据生成; 双系统架构。
- **Helix (Figure):** 人形 VLA, **system1/system2**——S2(7-9Hz 语义) + S1(200Hz 控制), 单网络控制上半身高自由度。
- **Genie / world models:** 交互式可玩世界模型, 生成可控环境。

Q65 ★ Diffusion Policy & ACT (必须默写, 详见 M4)

- **Diffusion Policy:** 条件去噪扩散生成动作序列, 建模多模态分布, receding horizon。
- **ACT:** Transformer CVAE + action chunking + temporal ensemble, ALOHA 精细操作。

Q66 ◆ 讲一篇你最近读的论文 (答题框架)

四段式, 务必准备 1-2 篇能深聊:

1. **Motivation:** 它要解决什么问题, 之前方法的缺陷。
2. **Method:** 核心方法 (一句话抓本质 + 关键设计)。
3. **Experiments:** 在什么 benchmark、和谁比、关键结论数字。
4. **批判:** 局限、你的质疑、可改进点、你会怎么 follow up。

加分: 能把它放进领域脉络 (“它是 *RT-2* 之后解决推理慢的尝试”), 并联系你自己的项目。

本模块自测**自测清单**

- ☐ RT-1/2/X 每步解决什么（多任务/语义/跨本体）
- ☐ OpenVLA 双编码器 + 关键结论
- ☐ π_0 的 flow matching action expert
- ☐ RDT 的 diffusion+ 大模型 + 双臂
- ☐ Octo 的模块化 generalist 设计
- ☐ 准备好 1-2 篇论文用四段式深聊

12 M11 · 工程与部署—逐题精讲

工业岗高频。核心矛盾：大模型 VLA 慢，机器人控制要快、要稳、算力受限。

Q67 ★ 大模型 VLA 推理慢，部署加速手段

- 模型压缩：
- 量化 (quantization)：FP16/INT8/INT4，OpenVLA 量化几乎不掉点。
- 蒸馏 (distillation)：大模型教小模型。
- 剪枝 (pruning)：去冗余参数/层。
- 减少计算量：
- visual token 压缩 (pooling/merging/Q-Former) ——VLA 大头在视觉 token。
- 更少采样步：diffusion→DDIM/一致性模型；用 flow matching 减步数。
- 推理系统：
- KV cache、FlashAttention、算子融合、TensorRT/ONNX 编译。
- action chunking 摊薄推理成本（一次推理管多步）。
- 异步推理：推理与执行解耦，不阻塞控制。
- 架构：分层——大模型低频、小网络/控制器高频。

Q68 action chunking / receding horizon 的工程权衡

- chunk 越长：推理频率越低、越省算力、动作越平滑；但对环境变化反应越慢（开环时间长，动态场景易失败）。
- chunk 越短：闭环反应快、纠错及时；但推理频繁、算力压力大、可能抖动。
- receding horizon：预测 H 步只执行前 k 步再重规划——平衡平滑与反应性。
- temporal ensemble：重叠预测加权平均，进一步平滑。
- 工程上按任务动态/精度需求调 H 和执行步数。

Q69 ★ 边缘设备 (Jetson) 算力受限怎么办

- 模型侧：量化/蒸馏出小尺寸 VLA，或大模型放云端、边缘跑轻量策略。
- 调度侧：分层——边缘高频控制，大模型低频（本地或云）。
- 系统侧：TensorRT 优化、半精度、batch=1 延迟优化、固定输入尺寸。
- 取舍：云-边协同（云端大模型规划，边缘实时控制）vs 全本地（低延迟、隐私、断网可用）。

Q70 实时控制对延迟/抖动的要求；大模型如何不阻塞控制回路

- 要求：控制回路要**确定性低延迟**，抖动（jitter）会破坏稳定性，尤其力控/平衡。
- 不阻塞做法：
- 控制回路独立**实时线程/RTOS**，固定周期运行。
- 大模型在**单独线程/进程异步推理**，输出动作 chunk 放共享缓冲区。
- 控制器从缓冲区取目标、插值到高频，永不等待大模型。
- 看门狗/安全层：大模型超时则用上一 chunk 或安全策略兜底。

Q71 ♦ 从数据到部署的完整 MLOps 闭环（开放题）

数据采集(遥操作/仿真) -> 数据管理与清洗(版本化、质检)
-> 训练(分布式、实验跟踪) -> 评测(仿真+真机, 自动化)
-> 部署(量化/编译, 边缘) -> 监控(成功率、失败案例回流)
-> 数据飞轮回到采集

- 关键点：**数据版本化**（可复现）、**自动化评测**（真机评测最贵最关键）、**失败案例回流**（闭环）、**A/B 与灰度部署**、**安全监控**。
- 强调：机器人 MLOps 比纯软件多了**真机评测**和**安全**两道难关。

本模块自测

自测清单

- ☐ VLA 加速三层：压缩/减计算/推理系统
- ☐ chunk 长短的权衡（平滑 vs 反应性）
- ☐ 边缘部署的云-边协同取舍
- ☐ 大模型如何异步不阻塞实时控制回路
- ☐ 画出 MLOps 闭环并指出真机评测/安全的特殊性

13 M12 · 开放题与项目面—逐题精讲

决定面试上限的部分。技术题看下限，开放题和项目面看上限——研究品味、工程判断、沟通。

Q72 ★ 介绍一个项目（STAR + 技术深度）

用 STAR 框架，且每层都能被深挖：

- **S(背景)**：什么问题、为什么重要、约束是什么。
- **T(任务)**：你的具体目标和你负责的部分（区分团队 vs 个人贡献）。
- **A(行动)**：方法选型与**为什么这么选**（对比了哪些方案）、关键技术难点、你怎么解决。
- **R(结果)**：量化结果（成功率 ↑X%、延迟 ↓Y）、和 baseline 对比、上线/影响。
- **复盘**：踩了什么坑、学到什么、还能怎么改进。

准备 2-3 个项目，深浅可控。最深的那个要能扛住”为什么不用 X””这里为什么 work””换个场景还行吗”连环追问。

Q73 ★ 相比 baseline 好在哪 + 再给两周怎么改进

- **好在哪**：找准真正的差异点（不是”我调了参”），用消融实验/对比数据支撑，讲清**机制层面**为什么更好。
- **怎么改进**：体现你有研究 roadmap：
 - 短期：更多数据/更强 backbone/加 chunking。
 - 中期：换更好的动作表示、引入 3D/触觉、Sim2Real。
 - 长期：scaling、泛化到新任务。
- 展示你不满足现状、有持续优化的思路。

Q74 ★ 未来 3 年技术路线：哪些泡沫，哪些真趋势

参考判断（给观点 + 理由，不要骑墙）：

- **真趋势**：数据规模化与数据飞轮、跨本体/通用策略、VLM+ 生成式动作（flow/diffusion）、分层 system1/system2、仿真 + 合成数据、世界模型辅助。
- **可能被高估**：纯端到端”一个大模型解决一切”短期落地、人形机器人量产时间表、纯视频生成直接当策略的实时性、Demo 的真实泛化能力。
- **关键变量**：数据是否突破、Sim2Real 是否被攻克、硬件成本与可靠性。

Q75 数据 vs 算法 vs 算力，瓶颈在哪

”我认为当前主瓶颈是**数据**——架构（VLA）已相对收敛，算力可堆，但缺乏互联网级机器人数据。其次是**评测**：真机评测慢且难复现，拖慢迭代。算法上仍有空间（动作表示、长程、泛化），但不是首要卡点。所以我赌‘数据采集效率 + Sim2Real + 评测自动化’是未来主战场。”

（与 M1-Q7 呼应，保持一致论述。）

Q76 ◆ Scaling law 在具身上成立吗

辩证回答：

- **支持**：RT-X/OpenVLA 显示数据多、跨本体共训性能涨，趋势上数据规模有效。
- **但有别于 NLP**：
 - 机器人数据**质量、分布、本体异构**影响巨大，不是单纯堆量。
 - 缺乏统一的“token 化”和无限数据源，scaling 曲线更陡更贵。
 - 动作的多模态/物理约束让单纯 next-token scaling 不直接适用。
- **结论**：“数据上去性能会涨，但回报和路径不同于 NLP——质量与多样性、Sim2Real、动作表示同样关键，光堆量不够。这也是为什么数据飞轮和仿真是核心。”

Q77 ◆ 从零做家庭服务机器人”大脑”的技术路线

结构化设计（展示系统思维）：

1. **架构**：分层 system2(VLM 语义规划/长程分解) + system1(学习型低频策略) + 底层控制器。
2. **策略**：VLA 端到端学操作技能（diffusion/flow 动作），LLM 做任务规划与人机交互。
3. **数据**：遥操作打底 + 仿真规模化 + 人类视频预训练，建数据飞轮。
4. **感知**：RGB-D/多视角 + 3D 表示，必要处加触觉。
5. **安全**：安全层兜底、力限、人在环、异常检测。
6. **部署**：云-边协同，边缘实时控制。
7. **评测**：仿真 + 真机自动化评测闭环。

强调取舍：先窄场景（厨房/收纳）做深，再扩展；可靠性和安全优先于花哨能力。

Q78 反问环节（问对问题 = 高情商 + 判断力）

可问面试官/团队：

- 团队当前最大的技术瓶颈是什么？数据/算法/工程哪块投入最多？
- 真机评测和数据采集是怎么组织的？数据飞轮转起来了吗？

- 端到端 vs 分层，团队的技术信仰是什么？
- 这个岗位 6-12 个月最希望我解决什么问题？
- 仿真和真机的比例？Sim2Real 现在是怎么做的？

好问题展示你懂行业真问题，也帮你判断团队是否靠谱。

本模块自测

自测清单

- ☐ 用 STAR 讲清一个项目，扛住连环追问
- ☐ 能说清”比 baseline 好在机制层面”+ 改进 roadmap
- ☐ 对技术趋势有观点（真趋势 vs 泡沫）
- ☐ 数据/算法/算力瓶颈判断（与 M1-Q7 一致）
- ☐ scaling law 辩证回答
- ☐ 准备 3-5 个高质量反问

14 深度补充 A · 数学基础速查（具身大模型）

面试常被追问”推导一下””为什么是这个公式”。本篇覆盖必须能手推的核心数学。记号： x 向量， $\odot$ 逐元素乘， $\|\cdot\|$ $L2$ 范数， E 期望， $\sim$ 采样自。

1. 注意力机制（Attention）

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- Q, K, V 由输入线性投影： $Q = XW_Q$ 等，形状 (n, d_k) 。
- 为什么除 $\sqrt{d_k}$ ：若 q, k 各维独立、均值 0 方差 1，则点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的方差为 d_k 。不缩放则 softmax 进入饱和区、梯度消失。除以 $\sqrt{d_k}$ 使方差回到 1。
- 复杂度 $O(n^2 d)$ ：这就是为什么 visual token 多了会爆（M2-Q11）。
- 多头：把 d 切成 h 份并行做注意力再拼接，让模型关注不同子空间。

2. 对比学习 / InfoNCE（CLIP 的数学）

一个 batch 有 N 个图文对。图像、文本各自编码并 **L2 归一化**得 $\{u_i\}, \{v_i\}$ 。相似度用余弦（归一化后即点积），温度 τ ：

$$\ell_i^{(i2t)} = -\log \frac{\exp(u_i \cdot v_i / \tau)}{\sum_{j=1}^N \exp(u_i \cdot v_j / \tau)}$$

对称地再算文本 $\rightarrow$ 图像 $\ell_i^{(t2i)}$ ，总损失：

$$\mathcal{L} = \frac{1}{2N} \sum_{i=1}^N (\ell_i^{(i2t)} + \ell_i^{(t2i)})$$

- 本质是 **N 分类的交叉熵**，正样本是对角线（label = i），负样本是 batch 内其它。
- 温度 τ ：小 $\rightarrow$ 分布尖锐、强调难负样本；CLIP 用可学习的 $\log(1/\tau)$ 并裁剪。
- 这是 M2-Q9。负样本越多（大 batch）效果越好。

3. 扩散模型（DDPM）——Diffusion Policy 的地基

前向加噪（固定，无参数），方差表 $\{\beta_t\}_{t=1}^T$ ，记 $\alpha_t = 1 - \beta_t$ ， $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ：

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, \beta_t I)$$

闭式（任意 t 一步到位，训练关键）：

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

训练目标（预测噪声，DDPM 简化损失）：

$$\mathcal{L} = \mathbb{E}_{x_0, \varepsilon, t} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$$

反向去噪采样（从 $x_T \sim \mathcal{N}(0, I)$ 迭代到 x_0 ）：

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

- 用在 Diffusion Policy：把 x 换成动作序列 $a_{t:t+H}$ ，网络额外条件于观测 o ： $\varepsilon_\theta(a^k, k, o)$ (k 是去噪步)。
- 与 score 的关系： $\nabla_x \log q(x_t) \approx -\varepsilon_\theta / \sqrt{1 - \alpha_t}$ ，所以“预测噪声” $\approx$ “学分数” (score matching)。
- 为什么能建模多模态动作分布 (M4-Q27)：扩散是从噪声采样的生成模型，能覆盖多峰，而 MSE 回归只给条件均值。
- 加速：DDIM（确定性、跳步）、一致性模型、或直接换 flow matching。

4. Flow Matching (π_0 的动作生成)

学一个速度场 $v_\theta(x, t)$ ，把噪声分布连续地“流”成数据分布。用条件最优传输路径（直线插值）， $x_0 \sim \mathcal{N}(0, I)$ 噪声、 x_1 数据：

$$x_t = (1 - t) x_0 + t x_1, \quad t \in [0, 1]$$

该直线路径的目标速度是常向量 $x_1 - x_0$ 。训练损失（回归速度）：

$$\mathcal{L} = \mathbb{E}_{t, x_0, x_1} \|v_\theta(x_t, t) - (x_1 - x_0)\|^2$$

采样：解 ODE $\frac{dx}{dt} = v_\theta(x, t)$ ，从 x_0 积分到 $t = 1$ （欧拉法几步即可）。

- 相比 diffusion：路径是直线、采样步数少 $\rightarrow$ 推理快 (M3-Q21)。
- 用在 VLA： x = 动作 chunk， v_θ 额外条件于观测/语言 token。

5. VAE / CVAE (ACT 的数学)

ELBO（证据下界）：

$$\log p(x) \geq \underbrace{\mathbb{E}_{q(z|x)} [\log p(x|z)]}_{\text{重建}} - \underbrace{\text{KL}(q(z|x) \parallel p(z))}_{\text{正则到先验}}$$

- 先验 $p(z) = \mathcal{N}(0, I)$ ；编码器输出 μ, σ 。

- **重参数化技巧** (让采样可导): $z = \mu + \sigma \odot \varepsilon, \varepsilon \sim \mathcal{N}(0, I)$ 。
- 两高斯 KL 闭式: $KL = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1)$ 。
- **CVAE (ACT)**: 编码器/解码器都条件于观测 o 。训练时 encoder 用动作序列推 z (捕捉人类示范多模态/风格), decoder $p(a_{t:t+H} | o, z)$ 重建; **推理时 $z = 0$ (先验均值)**, 只跑 decoder。 z 的作用就是吸收”同一观测下不同人的不同动作”这种多模态性, 避免回归取平均 (M4-Q26/Q27)。

6. 强化学习核心公式

回报与价值: $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}, V^\pi(s) = \mathbb{E}[G_t | s_t = s], Q^\pi(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a]$ 。

Bellman: $Q^\pi(s, a) = \mathbb{E}_{s'}[r + \gamma \mathbb{E}_{a' \sim \pi} Q^\pi(s', a')]$ 。

策略梯度 (REINFORCE/Actor-Critic):

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) A(s, a)], \quad A = Q - V \text{ (优势)}$$

GAE (优势估计): $\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 。

PPO 裁剪目标 (on-policy, M5-Q31): 令 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$,

$$L^{\text{CLIP}} = \mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

裁剪防止单步更新过大, 是 PPO 稳定的关键。

SAC 最大熵 (off-policy, 样本高效): 目标加熵项 $\mathbb{E}[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))]$, 鼓励探索, α 可自动调。

7. 行为克隆的复合误差界 (BC 为什么 $O(\epsilon T^2)$)

设策略在专家分布下每步犯错概率 $\leq \epsilon$ (0-1 损失)。一旦犯错就可能进入训练 never 见过的状态, 后续不再有保证。Ross & Bagnell 的结论:

$$J(\pi_{\text{BC}}) - J(\pi^*) = O(\epsilon T^2)$$

直觉: 第 t 步首次出错的概率 $\sim \epsilon$, 之后 $\sim (T-t)$ 步全程”自由落体”, 求和得 T^2 量级。**Dagger** 让训练分布覆盖策略自身访问的状态, 把界改善到 $O(\epsilon T)$ (M4-Q23/Q24)。这是”为什么要 action chunking / Dagger / 闭环”的理论根。

8. 旋转表示 (动作里最容易被深挖的数学)

机器人动作含姿态, 神经网络怎么表示旋转很关键。

表示	维度	优点	坑
欧拉角	3	直观	万向锁 (gimbal lock)、不连续
轴角/旋转向量	3	紧凑、指数映射	2π 处不连续
四元数	4	无万向锁、插值好 (slerp)	单位约束、 双重覆盖 $q \equiv -q$
旋转矩阵	9	无歧义	需正交约束
6D 表示	6	连续、适合网络回归	需 Gram-Schmidt 还原

- **为什么网络回归旋转推荐 6D** (Zhou et al. 2019): 欧拉角/四元数在 $SO(3)$ 上的映射**不连续**, 网络难拟合 (输出突变)。6D 表示取旋转矩阵前两列 $[a_1, a_2]$, 用 **Gram-Schmidt** 正交化还原:

$$b_1 = \frac{a_1}{\|a_1\|}, \quad b_2 = \frac{a_2 - (b_1 \cdot a_2)b_1}{\|a_2 - (b_1 \cdot a_2)b_1\|}, \quad b_3 = b_1 \times b_2, \quad R = [b_1 \ b_2 \ b_3]$$

这个映射连续, 回归误差更小。很多操作策略/抓取网络用它。

- **四元数双重覆盖**: q 和 $-q$ 表示同一旋转, 做回归 loss 时要取 $\min(\|q - \hat{q}\|, \|q + \hat{q}\|)$ 或用测地距离。
- **指数映射**: 轴角 ω (轴 $\times$ 角) 旋转矩阵 $R = \exp([\omega]_{\times})$ (Rodrigues 公式), $SE(3)$ /李代数在控制与位姿优化里常用。

9. 动作离散化 (tokenization) 的数学 (RT-2/OpenVLA)

把连续动作维度 $a \in [a_{\min}, a_{\max}]$ 均匀分成 B 个 bin:

$$\text{token}(a) = \left\lfloor \frac{a - a_{\min}}{a_{\max} - a_{\min}} \cdot B \right\rfloor, \quad \hat{a} = a_{\min} + \frac{\text{token} + 0.5}{B}(a_{\max} - a_{\min})$$

- 量化误差 $\leq \frac{a_{\max} - a_{\min}}{2B}$, 所以 bin 越多越精但词表越大。OpenVLA 用 256 bin, 并常用**分位数归一化** (按数据分布的 1%~99% 分位裁剪) 抵抗离群值。
- 训练就是对 token 做**交叉熵** (分类), 而非回归——这让动作能复用 LLM 的词表与自回归框架 (M3-Q16 路线 A)。

自测 (能手推/手画吗)

自测清单

- ☐ attention 为何除 $\sqrt{d_k}$ (方差论证)
- ☐ 写出 InfoNCE 对称损失
- ☐ DDPM 前向闭式 + 噪声预测损失 + 反向采样
- ☐ flow matching 的直线路径、目标速度、ODE 采样

- ☐ CVAE 的 ELBO + 重参数化 + ACT 里 z 的作用
- ☐ PPO 裁剪目标, r_t 是什么
- ☐ BC 为何 $O(\epsilon T^2)$, DAgger 为何 $O(\epsilon T)$
- ☐ 6D 旋转表示为何连续、Gram-Schmidt 还原

15 深度补充 B · 核心算法代码 (PyTorch)

面试可能让你”手写个 *loss* / 写个核心 *forward*”。以下都是**最小可读实现**，抓住骨架而非工程细节。统一约定：**B** = *batch*，**T/H** = 时间步，**A** = 动作维度，**D** = 特征维。

1. CLIP / InfoNCE 对比损失 (M2-Q9)

```
import torch, torch.nn.functional as F

def clip_loss(img_emb, txt_emb, logit_scale):
    # img_emb, txt_emb: (B, D); logit_scale = exp(可学习温度参数)
    img = F.normalize(img_emb, dim=-1)
    txt = F.normalize(txt_emb, dim=-1)
    logits = logit_scale * img @ txt.t()          # (B, B) 相似度矩阵
    labels = torch.arange(len(img), device=img.device) # 对角线为正样本
    loss_i2t = F.cross_entropy(logits, labels)
    loss_t2i = F.cross_entropy(logits.t(), labels)
    return (loss_i2t + loss_t2i) / 2
```

要点：归一化 → 温度缩放 → 对角线当 label → 对称交叉熵。

2. 动作离散化 / 反离散化 (RT-2、OpenVLA, M3-Q16 路线 A)

```
import torch

def tokenize_action(a, a_min, a_max, n_bins=256):
    # a: (... , A) 连续动作 -> (... , A) 离散 token id
    a = torch.clamp(a, a_min, a_max)
    norm = (a - a_min) / (a_max - a_min)          # [0,1]
    return torch.clamp((norm * n_bins).long(), 0, n_bins - 1)

def detokenize_action(tok, a_min, a_max, n_bins=256):
    norm = (tok.float() + 0.5) / n_bins           # bin 中心
    return a_min + norm * (a_max - a_min)

# 训练：对每个动作维度做分类交叉熵
# logits: (B, A, n_bins), target: (B, A) token id
def action_ce_loss(logits, target):
    return F.cross_entropy(logits.reshape(-1, logits.size(-1)),
                           target.reshape(-1))
```

实务：**a_min/a_max** 常用数据的 1%/99% 分位数（抗离群）。

3. Diffusion Policy ——训练步 + 采样 (M4-Q25)

```
import torch, torch.nn as nn, torch.nn.functional as F

class DiffusionPolicy(nn.Module):
    """eps_net(noisy_action_chunk, k, obs) -> 预测噪声"""
    def __init__(self, eps_net, betas):          # betas: (K,)
        super().__init__()
```

```

self.eps_net = eps_net
alphas = 1.0 - betas
self.register_buffer('betas', betas)
self.register_buffer('alphas', alphas)
self.register_buffer('abar', torch.cumprod(alphas, 0)) # _k

def loss(self, a0, obs):
    # a0: (B,H,A) 专家动作序列
    B, K = a0.size(0), self.betas.size(0)
    k = torch.randint(0, K, (B,), device=a0.device)
    eps = torch.randn_like(a0)
    abar = self.abar[k].view(B, 1, 1)
    a_k = abar.sqrt() * a0 + (1 - abar).sqrt() * eps # 前向闭式加噪
    eps_pred = self.eps_net(a_k, k, obs)
    return F.mse_loss(eps_pred, eps) # 预测噪声

@torch.no_grad()
def sample(self, obs, shape):
    # 反向去噪采样
    a = torch.randn(shape, device=obs.device) #  $x_T \sim N(0, I)$ 
    for k in reversed(range(self.betas.size(0))):
        kk = torch.full((shape[0],), k, device=obs.device)
        eps = self.eps_net(a, kk, obs)
        alpha, abar, beta = self.alphas[k], self.abar[k], self.betas[k]
        mean = (a - (1 - alpha) / (1 - abar).sqrt() * eps) / alpha.sqrt()
        a = mean + (beta.sqrt() * torch.randn_like(a) if k > 0 else 0)
    return a # (B,H,A) 动作序列
# 部署: 取前几步执行, receding horizon 再重采样

```

4. Flow Matching action head (π_0 思路, M3-Q21)

```

class FlowPolicy(nn.Module):
    """v_net(x_t, t, obs) -> 速度场"""
    def __init__(self, v_net):
        super().__init__(); self.v_net = v_net

    def loss(self, a1, obs):
        # a1: (B,H,A) 数据(专家动作)
        B = a1.size(0)
        a0 = torch.randn_like(a1) # 噪声
        t = torch.rand(B, 1, 1, device=a1.device) #  $t \sim U(0,1)$ 
        x_t = (1 - t) * a0 + t * a1 # 直线路径
        target = a1 - a0 # 目标速度(常向量)
        return F.mse_loss(self.v_net(x_t, t.view(B), obs), target)

    @torch.no_grad()
    def sample(self, obs, shape, steps=10):
        # 解 ODE:  $dx/dt = v$ 
        x = torch.randn(shape, device=obs.device)
        dt = 1.0 / steps
        for i in range(steps):
            t = torch.full((shape[0],), i * dt, device=obs.device)
            x = x + self.v_net(x, t, obs) * dt # 欧拉积分
        return x

```

对比 DiffusionPolicy: 训练目标从”预测噪声”变”回归速度”, 采样几步即可 → 更快。

5. ACT 的 CVAE (M4-Q26)


```

class ACTPolicy(nn.Module):
    def __init__(self, obs_enc, encoder, decoder, A, H, zdim=32):
        super().__init__()
        self.obs_enc = obs_enc          # 观测编码
        self.encoder = encoder           # (动作序列, obs) -> mu, logvar
        self.decoder = decoder          # (obs, z) -> 动作序列
        self.to_mu = nn.Linear(encoder.dim, zdim)
        self.to_lv = nn.Linear(encoder.dim, zdim)

    def forward(self, obs, a_gt=None):
        o = self.obs_enc(obs)
        if a_gt is not None:            # 训练: 用真值动作推 z
            h = self.encoder(a_gt, o)
            mu, logvar = self.to_mu(h), self.to_lv(h)
            z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu) # 重参数化
            a_pred = self.decoder(o, z)
            recon = F.l1_loss(a_pred, a_gt) # 重建(L1常用)
            kld = -0.5 * torch.mean(1 + logvar - mu**2 - logvar.exp())
            return a_pred, recon + 1e-2 * kld # beta*KL, beta小
        else:                            # 推理: z=0(先验均值)
            z = torch.zeros(o.size(0), self.to_mu.out_features, device=o.device)
            return self.decoder(o, z)
# temporal ensemble: 对重叠时间步的多次预测做指数加权平均(略)

```

6. PPO 损失 (M5-Q31)

```

def ppo_loss(logp, logp_old, adv, value, ret, entropy, clip=0.2, c_v=0.5, c_e=0.01):
    ratio = torch.exp(logp - logp_old) # r_t = pi/pi_old
    adv = (adv - adv.mean()) / (adv.std() + 1e-8) # 优势归一化
    l_clip = -torch.min(ratio * adv,
                        torch.clamp(ratio, 1 - clip, 1 + clip) * adv).mean()
    l_v = c_v * F.mse_loss(value, ret) # 价值回归
    l_e = -c_e * entropy.mean() # 熵奖励(鼓励探索)
    return l_clip + l_v + l_e

def compute_gae(rewards, values, gamma=0.99, lam=0.95):
    adv, gae = torch.zeros_like(rewards), 0
    for t in reversed(range(len(rewards))):
        nxt = values[t + 1] if t + 1 < len(values) else 0
        delta = rewards[t] + gamma * nxt - values[t]
        gae = delta + gamma * lam * gae
        adv[t] = gae
    return adv

```

7. 6D 旋转 → 旋转矩阵 (M3 动作姿态 / 数学篇 8)

```

def rot6d_to_matrix(x): # x: (B,6) 网络输出
    a1, a2 = x[:, :3], x[:, 3:]
    b1 = F.normalize(a1, dim=-1)
    a2 = a2 - (b1 * a2).sum(-1, keepdim=True) * b1 # 去掉在 b1 上的分量
    b2 = F.normalize(a2, dim=-1) # Gram-Schmidt
    b3 = torch.cross(b1, b2, dim=-1)
    return torch.stack([b1, b2, b3], dim=-1) # (B,3,3)

```

为什么不用欧拉角/四元数回归：它们到 $SO(3)$ 不连续，网络难拟合（数学篇 8）。

8. 一个极简 VLA forward（把上面串起来，M3-Q15）

```
class TinyVLA(nn.Module):
    def __init__(self, vision_enc, text_enc, llm, action_head):
        super().__init__()
        self.vision_enc = vision_enc      # 图像 -> visual tokens (B,Nv,D)
        self.text_enc = text_enc          # 指令 -> text tokens (B,Nt,D)
        self.llm = llm                    # Transformer 主干
        self.action_head = action_head    # 取最后隐状态 -> 动作(三选一表示)

    def forward(self, image, instruction, state=None):
        v = self.vision_enc(image)
        t = self.text_enc(instruction)
        tokens = torch.cat([v, t], dim=1)  # 多模态拼接(LLaVA式)
        if state is not None:
            tokens = torch.cat([tokens, state], dim=1) # 本体状态
        h = self.llm(tokens)               # (B, N, D)
        return self.action_head(h[:, -1])  # 动作: 离散token/diffusion/flow
```

action_head 三选一即对应 M3-Q16 三条路线：分类头 (离散) / MLP(回归) / Diffusion 或 Flow 头 (生成式)。

自测（能默写骨架吗）

自测清单

- ☐ InfoNCE：归一化 → 温度 → 对角 label → 对称 CE
- ☐ 动作离散化的 tokenize/detokenize
- ☐ Diffusion Policy：前向闭式加噪 + 预测噪声 loss + 反向采样
- ☐ Flow matching：直线路径 + 速度回归 + ODE 采样
- ☐ CVAE：训练推 z + 重参数化 + 重建 + KL，推理 $z=0$
- ☐ PPO：ratio + clip + 价值 + 熵
- ☐ 6D → 矩阵的 Gram-Schmidt